

RV32I 5-Stage Pipeline CPU 구현 정리

이번 작업에서는 RV32I CPU를 5-stage pipeline 구조로 나누고, branch taken 상황에서 잘못 들어온 instruction을 flush하는 기본 흐름까지 연결했다. 아직 forwarding과 load-use stall은 넣지 않았기 때문에 테스트 프로그램은 RAW hazard가 생기지 않도록 NOP을 포함해서 구성했다.

1. 구현 목표

구분	구현 내용
Pipeline 구조	IF, ID, EX, MEM, WB stage로 나누고 IF/ID, ID/EX, EX/MEM, MEM/WB register를 연결했다.
PC 제어	PC+4만 수행하던 구조에서 branch taken 시 branch target address로 이동하도록 PC.v를 분리했다.
Branch 처리	ALU에서 BEQ, BNE, BLT, BGE 조건을 판단하고, taken이면 IF/ID와 ID/EX를 flush하도록 연결했다.
검증 준비	branch taken/not-taken 흐름을 확인하기 위해 program.hex와 tb_branch_nohazard.v를 작성했다.

2. Pipeline 전체 흐름

Stage	주요 모듈	구현한 역할
IF	PC.v, imem.v	현재 PC로 instruction memory를 읽고, branch taken이면 next_PC를 branch target으로 바꾼다.
IF/ID	IF_ID_reg.v	fetch된 PC와 instruction을 ID stage로 넘기며, flush 시 NOP을 넣는다.
ID	Control.v, reg_file.v, imm_gen.v	opcode decode, register read, immediate 생성, control signal 생성을 담당한다.
ID/EX	ID_EX_reg.v	EX stage에서 필요한 operand와 control signal을 저장한다. flush 시 control을 0으로 만든다.
EX	alu_controller.v, alu.v, bta.v	ALU 연산, branch condition 판단, branch target address 계산을 수행한다.
EX/MEM	EX_MEM_reg.v	ALU 결과, store data, memory/writeback control을 MEM stage로 넘긴다.
MEM	dmem.v	LW/SW를 위한 word 단위 data memory 접근을 처리한다.
MEM/WB	MEM_WB_reg.v	load data와 ALU result를 WB stage로 넘긴다.
WB	cpu_core.v 내부 mux	memtoreg에 따라 register file에 쓸 값을 선택한다.

3. 모듈별 구현 내용

모듈	구현 내용	핵심 신호/포인트
cpu_core.v	전체 CPU top module이다. PC, instruction memory, control, register file, ALU, data memory, pipeline register를 모두 연결했다.	branch_taken = E_branch && E_branch_condition으로 flush 기준을 만들고, PC는 next_PC를 받아 갱신한다.
PC.v	next PC 선택을 별도 모듈로 분리했다. branch가 taken이면 bta를 선택하고, 아니면 PC+4를 선택한다.	next_PC = branch && branch_condition ? bta : PC + 4
imem.v	\$readmemh로 program.hex를 읽는 instruction memory이다. PC byte address에서 [11:2]를 사용해 word index로 접근한다.	INIT_FILE, DEPTH parameter 사용
IF_ID_reg.v	IF stage의 PC와 instruction을 ID stage로 넘긴다. reset이나 F_flush가 들어오면 NOP을 넣는다.	NOP = 32'h00000013

모듈	구현 내용	핵심 신호/포인트
Control.v	opcode를 보고 main control signal을 만든다. LW, SW, R-type, I-type, Branch, Lui, Jal, Jalr case를 나누었다.	alusrc, branch, aluop, memwrite, memread, memtoreg, regwrite
reg_file.v	32개 register 배열과 x0 read 처리를 구현했다. RS1/RS2에 대해 조합 read를 제공한다.	x0이면 항상 0을 출력하도록 처리
imm_gen.v	opcode에 따라 I, S, B, U, J immediate를 만든다. branch immediate는 LSB 0을 붙여 word alignment를 반영했다.	LW, SW, Branch, Lui, Jal, Jalr immediate 지원
ID_EX_reg.v	ID stage에서 만든 operand/control을 EX stage로 넘긴다. branch taken flush가 들어오면 bubble처럼 control과 data를 0으로 만든다.	i_flush 입력 추가
alu_controller.v	ALUOp와 funct를 보고 실제 ALU signal을 만든다. R-type과 I-type을 case로 분리했다.	ADD, SUB, AND, OR, XOR, SLL 계열 mapping
alu.v	ALUSrc에 따라 두 번째 operand를 register 또는 immediate로 선택하고 ALU 연산을 수행한다. branch 조건도 여기서 판단한다.	BEQ, BNE, BLT, BGE condition 출력
bta.v	branch target address를 계산한다. EX stage의 PC와 branch immediate를 더해 bta를 만든다.	$bta = E_PC + i_imm$
EX_MEM_reg.v	EX 결과를 MEM stage로 넘긴다. ALU 결과, store data, rd, memory/writeback control을 저장한다.	memread, memwrite, memtoreg, regwrite 전달
dmem.v	word 단위 data memory이다. ALU 결과를 주소로 사용하고, memread/memwrite에 따라 load/store를 수행한다.	word_addr = $i_alu_out[ADDR_WIDTH+1:2]$
MEM_WB_reg.v	MEM stage 결과를 WB stage로 넘긴다. ALU result와 load data를 저장하고 regwrite/rd도 같이 넘긴다.	WB_memtoreg로 writeback source 선택
program.hex	branch 검증용 instruction image이다. RAW hazard를 피하기 위해 중간에 NOP을 넣었다.	BEQ, BNE, BLT, BGE test sequence 포함
tb_branch_nohazard.v	branch flush 확인용 testbench이다. register 결과를 check_reg task로 비교하고 PASS/FAIL 메시지를 출력한다.	PASS: branch taken/not-taken flush test without RAW hazards

4. Control signal 정리

Instruction	ALUSrc	Branch	ALUOp	MemWrite	MemRead	MemToReg	RegWrite
LW	1	0	00	0	1	1	1
SW	1	0	00	1	0	0	0
R-type	0	0	10	0	0	0	1
I-type ALU	1	0	11	0	0	0	1
Branch	0	1	01	0	0	0	0
Lui	1	0	11	0	0	0	1
Jal	0	1	00	0	0	0	1
Jalr	1	1	00	0	0	0	1

5. ALU와 branch 조건

구분	case	구현 내용
ALUOp 00	load/store	주소 계산을 위해 ADD signal을 사용한다.
ALUOp 01	branch	기본적으로 SUB signal을 사용하고, 실제 branch condition은 funct3로 판단한다.

구분	case	구현 내용
ALUOp 10	R-type	funct4({funct7[5], funct3})로 ADD, SUB, AND, OR, XOR, SLL을 구분한다.
ALUOp 11	I-type	funct3로 ADDI, ANDI, ORI, XORI, SLLI를 구분한다.
BEQ	funct3=000	i_rdata1 == operand2이면 branch_condition=1
BNE	funct3=001	i_rdata1 != operand2이면 branch_condition=1
BLT	funct3=100	\$signed(i_rdata1) < \$signed(operand2)이면 branch_condition=1
BGE	funct3=101	\$signed(i_rdata1) >= \$signed(operand2)이면 branch_condition=1

6. Branch flush 흐름

단계	처리 내용
1	EX stage에서 ALU가 branch condition을 계산한다.
2	cpu_core.v에서 branch_taken = E_branch && E_branch_condition을 만든다.
3	PC.v는 branch_taken이면 bta를 next_PC로 선택한다.
4	IF_ID_reg.v는 F_flush가 들어오면 instruction을 NOP으로 바꾼다.
5	ID_EX_reg.v는 i_flush가 들어오면 control signal을 0으로 만들어 bubble을 넣는다.

```

assign branch_taken = E_branch && E_branch_condition;
assign next_PC = (branch && branch_condition) ? bta : PC + 32'd4;
assign bta = E_PC + i_imm;

```

7. 검증용 파일

파일	역할
program.hex	BEQ taken, BNE not taken, BLT taken, BGE taken 흐름을 확인하기 위한 instruction image
tb_branch_nohazard.v	register 결과를 check_reg task로 비교하고 branch flush PASS/FAIL을 출력하는 testbench

현재 test sequence는 forwarding과 stall unit이 아직 없다는 점을 고려해 RAW hazard가 생기지 않도록 NOP을 넣어서 구성했다. 따라서 이번 검증의 초점은 data hazard가 아니라 branch taken/not-taken과 flush 동작이다.

8. 추가 구현 필요 내용

우선순위	추가할 내용	이유
1	reg_file writeback block 활성화 상태 확인	현재 register file의 write always block은 최종 시뮬레이션 전에 활성화 여부를 반드시 확인해야 한다.
1	Forwarding unit	EX/MEM, MEM/WB 결과를 다음 instruction의 EX operand로 넘겨 RAW hazard를 줄이기 위해 필요하다.
1	Hazard detection unit	load-use hazard는 forwarding만으로 해결되지 않으므로 PC/IF_ID hold와 ID_EX bubble 처리가 필요하다.
1	Load/store 검증	dmem.v는 들어갔지만 LW/SW를 포함한 end-to-end test가 더 필요하다.
2	JAL/JALR datapath 완성	PC+4 writeback, JALR target 계산, target 하위 bit 정리까지 별도 처리가 필요하다.
2	WB source mux 확장	현재 memtoreg 1비트로는 ALU/MEM 외 PC+4, LUI immediate 같은 source를 모두 표현하기 어렵다.
2	RV32I ALU 연산 확장	SLT, SLTU, SRL, SRA 등 기본 RV32I 연산을 추가해야 한다.

우선순위	추가할 내용	이유
2	memory width 처리	LB/LBU/LH/LHU/SB/SH 지원을 위해 byte enable과 sign/zero extension이 필요하다.
3	MMIO bus	나중에 A* accelerator나 peripheral을 붙이려면 address decode 기반 memory-mapped I/O가 필요하다.
3	A* accelerator interface	start, done, result register를 통해 CPU가 accelerator를 제어하는 구조가 필요하다.

9. 정리

이번 구현에서는 5-stage pipeline CPU의 기본 구조를 만들고, branch taken 시 PC redirect와 pipeline flush가 일어나도록 연결했다. PC 선택, branch target 계산, IF/ID NOP flush, ID/EX bubble flush를 분리해 두었기 때문에 이후 forwarding과 hazard detection을 추가하기 좋은 구조가 되었다.